Nested Profile-Likelihood Inference of (A_s, A_c) from the Full Pixel Image

`python-scripts/joint_profile_inference.py`

Contents

1 Purpose and Relationship to Other Estimators

This note documents `joint_profile_inference.py`, an estimator of the gradiometer signal $\beta = (A_s, A_c)$ (notation as in `notes/image_likelihood.tex`) that profiles *every* per-shot nuisance parameter — the common laser phase ϕ_i and both clouds' 8-dimensional state $\eta_{i,z=0}, \eta_{i,z=100}$ — directly against the full pixel-binned Poisson likelihood, jointly with the outer optimisation over β, rather than against a reduced summary (port counts or image moments).

Why this exists. `crb_signal.py`'s Cramér–Rao analysis showed that the port-summed/moment-based pipeline (`map_inference.py`) retains only $\sim$9% of the Fisher information available in ϕ_i once realistic cloud-parameter uncertainty is propagated, whereas the full pixel-resolved likelihood retains $\sim$96% (each AI's (ϕ, η) Fisher block goes from rank 1, in the port-summed reduction, to full rank 9 once individual pixel counts are used — see `crb_signal.pixel_fisher_block`). This script is the estimator designed to realise that gain in an actual per-shot fit, not just in a Fisher-information forecast.

Relationship to other pipelines in this repo.

- `map_inference.py` profiles a 4-moment Kalman summary of η_{iz} and marginalises ϕ_i on a grid (Section 2 of `notes/image_likelihood.tex`). Cheap, but discards spatial information beyond the first two moments.
- `full_image_is_inference.py` uses the full pixel image but marginalises η by importance sampling from a moment-posterior proposal (Section 7 of `notes/image_likelihood.tex`).
- **This script** uses the full pixel image and *profiles* (point-estimates, does not marginalise) η_{iz} *and* ϕ_i jointly, re-solving both at every trial β. It is the most computationally direct realisation of the full-information estimate, at the cost of the heaviest per-evaluation optimisation.

2 Per-Shot Pixel Likelihood (Recap)

This section restates, without re-deriving, the model shared with `notes/image_likelihood.tex` (Sections 1–3 there), fixing notation for what follows.

2.1 Parameters

- $\beta = (A_s, A_c)$: the two signal amplitudes (science parameters).
- ϕ_i: common laser phase for shot i, shared by both AIs, a priori Uniform$(0, 2\pi)$.
- $\delta\phi_i(\beta) = A_s \sin(2\pi f t_i) + A_c \cos(2\pi f t_i)$: the injected differential phase at AI $z = 100$ (zero at $z = 0$).
- $\theta_{iz} = (\mu_{x0}, \mu_{y0}, \mu_{v_x}, \mu_{v_y}, \sigma_{x0}, \sigma_{y0}, \sigma_{v_x}, \sigma_{v_y})_{iz} \in \mathbb{R}^8$: the initial phase-space Gaussian cloud parameters for AI $z \in \{0, 100\}$ in shot i (called `eta` elsewhere in the repo; renamed θ here to match the source file's variable names).

The full per-shot parameter vector profiled by this script is

$$x_i = (\phi_i,\ \theta_{i,0},\ \theta_{i,100}) \in \mathbb{R}^{17}. \tag{1}$$

2.2 Pixel Rate and Poisson Likelihood

For AI z, state $s \in \{g, e\}$ (ground/bright, excited/dark port group), pixel b:

$$\lambda_{izsb} = L_{iz}\big[A_{zsb}(\theta_{iz}) + C^c_{zsb}(\theta_{iz}) \cos\Phi_{iz} + C^s_{zsb}(\theta_{iz}) \sin\Phi_{iz}\big], \qquad \Phi_{iz} = \phi_i + \delta\phi_i(\beta)\,[z{=}100], \tag{2}$$

with $n_{izsb} \sim \text{Poisson}(\lambda_{izsb})$ and L_{iz} the atom-number scale factor (plug-in normalisation matching predicted to observed total counts, Eq. 4 below; *not* a free parameter). The code's variable names for (A, C^c, C^s) per state are `A_g,Cc_g,Cs_g` (state g) and `A_e,Cc_e,Cs_e` (state e), returned as `(n_bins,)` arrays by `acs_obj.pixel_acs(theta)` (Section 3).

The per-shot, per-AI negative log-likelihood implemented in `_ai_nll_only`/`_ai_nll_and_grad_analytic` is

$$-\log \mathcal{L}_{iz}(\theta_{iz}, \Phi_{iz}) = \sum_b \Big[\mu_{gb} - n_{gb} \log \mu_{gb}\Big] + \sum_b \Big[\mu_{eb} - n_{eb} \log \mu_{eb}\Big], \qquad \mu_{sb} \equiv \lambda_{izsb}, \tag{3}$$

(constants $\log n!$ dropped), with the scale factor computed *inline, at every evaluation*, not cached from a separate profiling pass:

$$L_{iz} = \frac{\sum_b (n_{gb} + n_{eb})}{\max\big(\sum_b (I_{gb} + I_{eb}),\, \varepsilon\big)}, \qquad I_{sb} = A_{sb} + C^c_{sb} \cos\Phi_{iz} + C^s_{sb} \sin\Phi_{iz}, \tag{4}$$

$\varepsilon = 10^{-300}$ (`crb.EPS`). Because L_{iz} is recomputed from (A, C^c, C^s) at the *current* (θ, Φ) every call, it is implicitly profiled out analytically at every point visited by the optimiser, not held fixed at a reference point as in the moment pipeline's $\hat{\eta}$-then-fix convention.

The joint per-shot objective actually minimised (`joint_nll/joint_nll_and_grad_analytic`) is

$$\boxed{F_i(x_i;\, \delta\phi_i^{\text{trial}}) = -\log \mathcal{L}_{i,0}(\theta_{i,0}, \phi_i) - \log \mathcal{L}_{i,100}(\theta_{i,100}, \phi_i + \delta\phi_i^{\text{trial}}) + \frac{1}{2}\left\|\tfrac{\theta_{i,0} - \bar{\theta}}{\sigma_\theta}\right\|^2 + \frac{1}{2}\left\|\tfrac{\theta_{i,100} - \bar{\theta}}{\sigma_\theta}\right\|^2,} \tag{5}$$

where $\bar{\theta}, \sigma_\theta$ are the (identical, independent) Gaussian prior mean/std for both AIs' clouds (`prior_mean`, `prior_std` arguments; matches the dataset's generation hyperparameters, van Trees / Bayesian-profiling convention, same as Section 2.3 of `notes/image_likelihood.tex`). ϕ_i carries *no* prior term (flat), consistent with $\phi_i \sim \text{Uniform}(0, 2\pi)$ — but note that here ϕ_i is *profiled* (point-estimated) jointly with θ, not *marginalised* over a phase grid as in `map_inference.py/full_image_is`. This is a genuine modelling difference, not just an implementation shortcut: see §9.

3 Pixel ACS via Exact CDF × 2D Gauss–Hermite

$(A_{zsb}, C^c_{zsb}, C^s_{zsb})$ are computed by `SemiAnalyticPixelACS.pixel_acs` (`profile_cloud_nuisances.py`), identical to the evaluator already documented in Section 7.3 of `notes/image_likelihood.tex`; restated compactly here since it is the main cost driver.

1. **Detection-plane Gaussian.** Free ballistic flight for time $T = t_{\text{det}}$:

$$\mu_{xf} = \mu_{x_0} + T\mu_{v_x}, \qquad \sigma^2_{xf} = \sigma^2_{x_0} + (T\sigma_{v_x})^2, \tag{6}$$

   and identically for y.

2. **Exact spatial weight.** Bin occupancy probability via the Gaussian CDF (no quadrature error, never zero):

$$P_b = \big[\Phi\big(\tfrac{x_{b,\text{hi}} - \mu_{xf}}{\sigma_{xf}}\big) - \Phi\big(\tfrac{x_{b,\text{lo}} - \mu_{xf}}{\sigma_{xf}}\big)\big] \times \big[y\text{-analogue}\big]. \tag{7}$$

3. **Conditional velocity distribution at the bin centre** (x_b, y_b) (standard Gaussian conditioning, decoupled in x/y):

$$v_{x_0} \mid x_b \sim \mathcal{N}\Big(\mu_{v_x} + \tfrac{T\sigma^2_{v_x}}{\sigma^2_{xf}}(x_b - \mu_{xf}),\ \tfrac{\sigma^2_{v_x}\sigma^2_{x_0}}{\sigma^2_{xf}}\Big), \tag{8}$$

   and identically for $v_{y_0} \mid y_b$.

4. **2D Gauss–Hermite quadrature** (n_{gh} nodes per velocity dimension, $n_v = n_{\text{gh}}^2$ nodes total per bin) over this conditional distribution gives the initial phase-space sample points $(x_0, y_0, v_{x_0}, v_{y_0})$ fed to the PSMAP lookup (§4), one full grid of n_v nodes *per bin*, processed `chunk_bins` bins at a time to bound GPU memory.

5. **Port sum and GH average.** PSMAP outputs ($\Delta\phi_p$, a_{0p}, a_{1p}) per port p (§4) give

$$A_p = a_{0p}^2 + a_{1p}^2, \quad C_p^c = \iota_p \, 2a_{0p}a_{1p}\cos\Delta\phi_p, \quad C_p^s = -\iota_p \, 2a_{0p}a_{1p}\sin\Delta\phi_p, \tag{9}$$

($\iota_p =$ `port_inter`, an interference sign/weight per port), summed over the ports belonging to state s (boolean masks `s0_g`, `s1_g`), GH-averaged over the n_v velocity nodes with weights w_2, and multiplied by P_b:

$$A_{zsb} = P_b \sum_{v=1}^{n_v} w_{2v} \sum_{p \in s} A_p(\text{node } v), \qquad (\text{and likewise for } C_{zsb}^c, C_{zsb}^s). \tag{10}$$

Default $n_{\text{gh}} = 20$ in `profile_cloud_nuisances.py`'s own CLI, but `joint_profile_inference.py` passes it through as `pixel_n_gh` with a much smaller default of **4** ($n_v = 16$ nodes/bin) — see §11 for the cost/accuracy trade-off this implies.

4 PSMAP Lookup: Catmull–Rom Interpolation and its Analytic Derivative

This section is *not* covered elsewhere in the notes tree and is specific to the analytic-gradient machinery (`pixel_acs_grad.py`, `pixel_acs_grad_batch.py`) built for this estimator.

4.1 Value: 4D Cubic Interpolation

The precomputed PSMAP is a 4D grid over (x, y, v_x, v_y), one grid per "port" $p = 1, \ldots, n_P$ (a port being one interfering/non-interfering branch of the atom-interferometer sequence), storing the accumulated phase $\Delta\phi_p$ and two beam-splitter amplitudes a_{0p}, a_{1p}. For a query point $(x_0, y_0, v_{x_0}, v_{y_0})$, each of the 4 coordinates is located in its grid cell with a fractional offset $t \in [0, 1]$, and the interpolated value is a **Catmull–Rom cubic** tensor product over the $4^4 = 256$ surrounding corners:

$$\hat{f}(x_0, y_0, v_{x_0}, v_{y_0}) = \sum_{k_x, k_y, k_{vx}, k_{vy}=0}^{3} w_x(t_x)_{k_x} \, w_y(t_y)_{k_y} \, w_{v_x}(t_{v_x})_{k_{vx}} \, w_{v_y}(t_{v_y})_{k_{vy}} \, f[\text{corner}], \tag{11}$$

where each $w(\cdot)_k$, $k = 0, \ldots, 3$ (stencil offsets $-1, 0, 1, 2$ from the cell floor), is the standard Catmull–Rom cubic-polynomial weight in t. This is the same interpolation scheme as `SurrogatePixelACS._eval_fast` documented for `PSMAPSurrogate` generally; what is new here is the gradient.

4.2 Analytic Gradient

Because each $w(t)_k$ is an explicit cubic polynomial in t, its derivative $w'(t)_k$ is also a closed-form polynomial (`_cr_weights_deriv`), so

$$\frac{\partial \hat{f}}{\partial x_0} = \frac{1}{\Delta x} \sum_{\text{corners}} w_x'(t_x)_{k_x} \, w_y(t_y)_{k_y} \, w_{v_x}(t_{v_x})_{k_{vx}} \, w_{v_y}(t_{v_y})_{k_{vy}} \, f[\text{corner}], \tag{12}$$

and symmetrically for y_0, v_{x_0}, v_{y_0} — an *exact* derivative of the interpolant, not a finite-difference approximation, at a cost of one extra weighted gather per coordinate (4 total) rather than re-touching the expensive per-port stacked-grid gather with perturbed inputs.

Two implementation details, both load-bearing for correctness:

- **Boundary clipping.** `_find_cell` clips both the cell index and the fractional coordinate t at the grid edge, so the interpolated *value* is pinned flat outside the domain (extrapolation-by-constant). The raw polynomial derivative does not know about this and would return a spurious nonzero slope there; it is explicitly zeroed for out-of-bounds points (`in_x`, `in_y`, `in_vx`, `in_vy` masks) to match the true (flat) value function.
- **Catastrophic cancellation in $\Delta\phi$.** The phase field $\Delta\phi_p$ is an unwrapped accumulated phase, $O(10^8)$ in magnitude, unlike $a_0, a_1 = O(1)$. The 256 corner weights used for a *derivative* sum to exactly zero in exact arithmetic, so summing $w'_k\,\Delta\phi[\text{corner}_k]$ directly loses essentially all significant digits to cancellation. Subtracting a common per-query reference value from all 256 corners before the weighted sum (`rebase=True` in `_gather_sum`) is mathematically identical (the weights still sum to zero, so a constant shift is annihilated) but removes the cancellation. This rebasing is applied *only* to the four derivative gathers, not the value gather.

4.3 Chaining Through to θ

The map $\theta \to (x_0, y_0, v_{x_0}, v_{y_0})$ (cloud parameters to PSMAP sample points, §3, steps 1–3) is pure elementary arithmetic — it never touches the PSMAP lookup — so its Jacobian is obtained by *cheap* central finite differences (step h_θ, hardcoded 10^{-9} in the batched path, 10^{-7} in an unused/dead local variable in `fit_beta` — see §9) and chained with the analytic $\partial\hat{f}/\partial(x_0, y_0, v_{x_0}, v_{y_0})$ from Eq. 12 via the chain rule. This hybrid (analytic-through-the-expensive-part, finite-difference-through-the-cheap-part) is what `pixel_acs_and_grad/pixel_acs_and_grad_bat` compute, returning both the 6-tuple $(A_g, C_g^c, C_g^s, A_e, C_e^c, C_e^s)$ and its $(8, 6, n_{\text{bins}})$ (single-shot) or $(N, 8, 6, n_{\text{bins}})$ (batched) gradient w.r.t. θ. $\partial/\partial\phi$ is fully analytic (plain $\cos$ / $\sin$ chain rule on Eq. 2), since ϕ enters the model only trigonometrically and never touches the PSMAP call.

5 Nested Profiling: Arrow Sparsity

Collect all shots' nuisances and the signal into one vector $(\beta,\, x_1, \ldots, x_{N_{\text{shots}}}) \in \mathbb{R}^{2+17N}$. Because shot i's likelihood depends on x_i and β only (never on x_j, $j \neq i$), the joint Fisher/Hessian matrix has exact **arrow sparsity**: the $(2+17N) \times (2+17N)$ matrix is dense only in the leading 2×2 block and the first two rows/columns; every x_i block is block-diagonal and couples *only* to β, never to another shot.

This licenses an exact reduction (not an additional approximation) of the joint MLE to:

> **outer** step on β (2 parameters), each evaluation requiring an **inner** per-shot solve of $x_i = (\phi_i, \theta_{i,0}, \theta_{i,100})$ (17 parameters, N_{shots} independent, embarrassingly parallel problems), warm-started from the previous outer iteration's solution.

This is explicitly *not* the same as profiling θ once at a reference β and holding it fixed (valid for the moment pipeline, Section 2.4 of `notes/image_likelihood.tex`, because η is only weakly coupled to β there): the module docstring records that θ (especially the weakly-identified ridge/width directions, Section 4.2 of `notes/image_likelihood.tex`) was empirically observed to shift by 6–24 posterior-σ as β sweeps a realistic range, so θ *must* be re-profiled at (or near) every trial β visited by the outer optimiser — hence the nested structure rather than a two-pass profile-then-fix scheme.

6 Inner Solve: Per-Shot $(\phi_i, \theta_{i,0}, \theta_{i,100})$

Given a trial $\delta\phi_i^{\text{trial}} = \delta\phi_i(\beta)$ and a warm-start $x_i^{(0)}$ (from the previous outer iteration; initialised at $(\phi = 0, \theta_{i,0} = \bar{\theta}, \theta_{i,100} = \bar{\theta})$ on the very first outer evaluation), the inner solve is

$$\hat{x}_i(\beta) = \arg\min_{x_i} F_i(x_i; \delta\phi_i^{\text{trial}}) \tag{13}$$

via bounded L-BFGS-B (`scipy.optimize.minimize`). Two implementations exist, differing only in how the gradient is supplied:

- `profile_shot` (`joint_nll`): scipy's own internal finite-difference gradient. Slower, but with "much less risk of a subtle sign/indexing bug" (module docstring) — the deliberately simple reference path. **No bounds are passed** to this `minimize` call (an asymmetry with the analytic path below; see §9).
- `profile_shot_analytic` (`joint_nll_and_grad_analytic`): the hybrid analytic gradient of §4, supplied via `jac=True`.

6.1 Bounds (analytic path only)

$$\phi_i \in [-50, 50] \text{ rad (effectively unconstrained)}, \qquad \theta_{iz,k} \in \bar{\theta}_k \pm 20\,\sigma_{\theta,k} \quad \forall k. \tag{14}$$

These are numerical safety rails, not part of the statistical model: an unbounded solve was observed directly to diverge (gradient norm *growing* with more iterations rather than shrinking) by chasing a spurious unbounded-improvement direction, motivating the generous but finite θ bound.

6.2 Objective Rescaling

Per-shot NLL is $O(10^7)$ (high photon counts), so raw gradients are $O(10^5$–$10^8)$, well outside the regime L-BFGS-B's default step-size heuristics (tuned for $O(1)$ objectives) are designed for. The objective and gradient are both rescaled by a constant $c = 1/\max(\|g_{\text{probe}}\|, 1)$, computed once (cached in `_scale_cache`) from the gradient at the very first call, before the optimiser sees them. This is a pure rescaling — it does not move the argmin — but is essential for L-BFGS-B's step-size selection to behave sanely.

6.3 Convergence Check and the Envelope-Theorem Shortcut

After `n_iter` L-BFGS-B iterations, the *unscaled* gradient at the reported solution is recomputed and its norm compared against `grad_tol` $\times \max(|\text{NLL}|, 1)$; if this is exceeded, a `UserWarning` is raised (*"inner solve NOT converged ... envelope-theorem outer gradient is unreliable here"*).

This check exists because `profile_shot_analytic` also returns

$$\left.\frac{\partial \text{NLL}_{i,100}}{\partial\, \delta\phi_i^{\text{trial}}}\right|_{\hat{x}_i}, \tag{15}$$

which, **by the envelope theorem, equals** $\partial F_i / \partial\, \delta\phi_i^{\text{trial}}$ exactly *at a true stationary point* of the inner objective (the z=0 contribution and all implicit $\partial \hat{x}_i / \partial\, \delta\phi_i^{\text{trial}}$ terms vanish there). This is what lets the outer gradient be computed *without* differentiating through the inner `arg min` — but the identity is only exact at convergence, so the diagnostic in this subsection is not optional decoration: an unconverged inner solve silently produces a biased outer gradient with no other symptom.

7 Outer Solve: $\beta = (A_s, A_c)$

Bounded L-BFGS-B over $\beta \in [-\texttt{grid\_half}, \texttt{grid\_half}]^2$ (default $\texttt{grid\_half} = 0.2\,\text{rad}$), multi-start from $\{(0,0)\} \cup \{(0.05, 0), (-0.05, 0.05)\}[:\texttt{n\_starts}-1]$, best of the `n_starts` local solutions kept. **Two outer implementations exist**, and they are *not* structurally symmetric:

7.1 `fit_beta` — Per-Shot Python Loop

For each trial β: loop over shots, calling `profile_shot` or `profile_shot_analytic` once per shot (warm-started from `state['x'][k]`, updated in place), summing the per-shot NLL.

- `use_analytic=False`: `neg_logL` alone, no `jac=True` — scipy finite-differences the *outer* objective too (on top of the FD inner solves via `profile_shot`).
- `use_analytic=True` (default): `neg_logL_and_grad_analytic`, which additionally accumulates the outer gradient via the envelope theorem (§6.3):

$$\frac{\partial(-\log\mathcal{L})}{\partial A_s} = \sum_i \frac{\partial \text{NLL}_{i,100}}{\partial\, \delta\phi_i} \sin(2\pi f t_i), \qquad \frac{\partial(-\log\mathcal{L})}{\partial A_c} = \sum_i \frac{\partial \text{NLL}_{i,100}}{\partial\, \delta\phi_i} \cos(2\pi f t_i), \quad (16)$$

  supplied via `jac=True`. The outer objective/gradient are additionally rescaled by a constant `obj_scale`, chosen once (from the raw gradient norm at $\beta = 0$) so that L-BFGS-B's first step is $O(0.02)$ in β-space rather than overshooting by orders of magnitude (same rationale as the inner rescaling, applied at the outer level too).

7.2 `fit_beta_batch` — Batched/GPU Inner Solve

Instead of a Python loop, all N shots' x_i are stacked into one $(17N,)$ vector and solved with a *single* L-BFGS-B call per outer β evaluation (`batched_nll_grad`, using `pixel_acs_grad_batch.pixel_acs_and_gra` §8), amortising GPU kernel-launch overhead across shots.

Critically, `fit_beta_batch`'s outer `minimize` call

```
minimize(neg_logL, b0, method='L-BFGS-B', bounds=bounds, options={'maxiter':
                                outer_maxiter})
```

had **no `jac=True` and no objective rescaling**: unlike `fit_beta`, there was no envelope-theorem outer gradient and no `obj_scale` analogue, so scipy finite-differenced the *outer* objective itself, with each FD probe triggering a fresh, warm-started, only-`n_inner_iter`-step inner batched solve. **This has since been fixed** (§10): `batched_nll_grad` now also returns the per-shot $\partial\text{NLL}_{100}/\partial\phi$ needed for the envelope theorem, and `fit_beta_batch` mirrors `fit_beta`'s analytic outer gradient, objective rescaling, and theta bounds. A second, structural problem remains unfixed — see §10.

8 Batching (`pixel_acs_grad_batch.py`)

"Batching" here means stacking the leading axis of every per-shot quantity so that the four `_eval_fast_grad` calls (one per grid dimension's derivative gather, §4) and the finite-difference θ-Jacobian loop are each issued *once* across all N shots simultaneously, rather than once per shot in a Python loop.

Quantity	Single-shot (`pixel_acs_grad.py`)	Batched (`pixel_acs_grad_batch.py`)
θ	$(8,)$	$(N, 8)$
Sample points $x_0, y_0, v_{x_0}, v_{y_0}$	$(n_{\text{bins}} n_v,)$	$(N,\ n_{\text{bins}} n_v)$
PSMAP lookup input (raveled)	$(n_{\text{bins}} n_v,)$	$(N\, n_{\text{bins}} n_v,)$ — one call
Base ACS $(A_g, \ldots, C_e^s)$	$6 \times (n_{\text{bins}},)$	$6 \times (N, n_{\text{bins}})$
θ-gradient	$(8, 6, n_{\text{bins}})$	$(N, 8, 6, n_{\text{bins}})$

The θ-Jacobian's finite-difference loop (`for k in range(8)`) still perturbs *one component at a time*, but perturbs it for *all N shots simultaneously* in that iteration (`tp[:, k] += h_theta[k]`, a scalar step broadcast across the shot axis) — so it is still 8 sequential perturbation calls total, not $8N$, each itself a batched (all-shots) `_sample_points_batch` call. The finish/port-sum/GH-average tail (`_finish_batch`) is likewise a straight axis-broadcast of the single-shot `_finish` logic, with an explicit `(N, n_bins, n_v)` reshape inserted before the GH weight contraction.

The module's own docstring flags this file as *"the highest-risk part of the session's speedup work, validated step by step against the already-correct per-shot loop before trusting it"* — no such validation script currently exists in the repo (as of this writing); see §10.

9 Assumptions and Simplifications

- ϕ_i **is profiled, not marginalised.** Unlike `map_inference.py`/`full_image_is_inference.py`, which integrate out ϕ_i over a phase grid (Section 2.3 of `notes/image_likelihood.tex`) precisely to avoid the per-shot phase absorbing the signal and biasing $\hat{\beta}$ toward zero, this estimator point-estimates ϕ_i jointly with θ. This is a genuine modelling choice (not just cost-driven), and its effect on $\hat{\beta}$'s bias/variance relative to the marginalised estimators has not yet been characterised in this file or its notes — flagged as an open question, not an established equivalence.
- L_{iz} **(flux normalisation) is analytically profiled at every evaluation**, not a free optimised parameter and not cached from a separate pass (§2).
- **Independent Gaussian priors** on $\theta_{i,0}$ and $\theta_{i,100}$, matching the dataset's generation hyperparameters, with *no cross-AI covariance term.*
- **Per-shot inner objective assumed effectively unimodal.** There is no multi-start at the per-shot (inner) level, only at the outer β level; the warm-start scheme also implicitly assumes the optimum moves continuously (and slowly, relative to `n_inner_newton`/`n_inner_iter` steps) as β is perturbed by the optimiser or by finite differences.
- **Bound constraints are numerical rails, not modelling choices** (§6).
- **Known inconsistencies in the source, noted for reproducibility:**
  - `fit_beta`'s `gh_order` parameter (and the matching `--gh_order` CLI flag) is accepted but **never used** inside `fit_beta` — it is a leftover from the port-summed API in `crb_signal.py` and has no effect on this script's behaviour.
  - `fit_beta` defines a local `h_theta = np.full(8, 1e-7)` that is **never passed anywhere** (dead code); the finite-difference step actually used for the analytic θ gradient is the module-level constant `_H_THETA_ANALYTIC = np.full(8, 1e-9)` in `pixel_acs_grad.py`. `fit_beta_batch` does define and use its own $h_\theta = 10^{-9}$ correctly.
  - `profile_shot` (the plain finite-difference inner solve) passes **no** `bounds` to `scipy.optimize.minimiz` (default: unconstrained), whereas `profile_shot_analytic` always bounds θ and ϕ

(§6). The two inner-solve implementations are therefore not drop-in equivalent even before considering gradient source.

10 Known Open Issues / Current Investigation

`fit_beta_batch` was reported to run faster than `fit_beta` but converge to a different $\hat{\beta}$. Investigation resolved this into **five distinct issues**, four now resolved or downgraded to non-issues, one (Issue 4) still genuinely open.

Issue 1 — fixed. Direct numerical cross-validation of `pixel_acs_grad_batch.py` against the per-shot loop (`pixel_acs_grad.py`), for several random shots and θ values, showed the batched value *and* gradient are **byte-identical** (max abs. difference = 0.0) to the single-shot reference — the batched pixel-level math in §3–4 is **not** the source of the discrepancy, despite the module docstring's own admission that this had never actually been checked. The real bug was exactly hypothesis (2) originally raised in this section: `fit_beta_batch`'s outer `minimize` call had no analytic gradient, so scipy's own finite-difference outer gradient was computed by perturbing β and re-solving the warm-started (`state['x']` mutated in place) inner problem at each FD probe — making repeated evaluations near the same β history-dependent and corrupting the FD gradient estimate. This has been **fixed**: `batched_nll_grad` now additionally returns the per-shot $\partial \mathrm{NLL}_{100}/\partial\phi$, `fit_beta_batch` now assembles the same envelope-theorem outer gradient as `fit_beta` (§7) and applies matching objective rescaling and θ/ϕ bounds on its inner solve.

Issue 2 — fixed. Forcing the inner batched solve to hard convergence originally revealed that the single monolithic $(17N)$-dimensional L-BFGS-B call converges markedly *worse* than N independent per-shot 9-dimensional solves, because bundling all shots into one L-BFGS-B call shares curvature/step-size history across shots whose ϕ-landscapes are independently multi-modal, so one "hard" shot could stall the whole batch — defeating the arrow-sparsity argument of §5, which licenses solving the shots *independently* given β. This is now **fixed** by `solve_inner_batch_independent`: a hand-vectorised per-shot L-BFGS (own curvature history, own step size, own convergence decision per shot), batching only the expensive value/gradient evaluation via `_batched_nll_grad_per_shot` – see that function's docstring for the three-attempt design history (naive BB, per-shot L-BFGS with per-shot backtracking retries, and the current one-eval-per-outer-iteration version, which is the only one that doesn't regress with N).

Re-validated in this session: an isolated per-shot comparison at matched convergence budgets (no outer loop, fixed trial β) shows `solve_inner_batch_independent` and `profile_shot_analytic` (scipy's own L-BFGS-B) land on the *same* solution for all 8 test shots — matching ϕ modulo 2π and matching NLL to within $\mathcal{O}(10^{-4})$ relative. An earlier full `fit_beta` vs `fit_beta_batch` comparison (`_validate_multistart.py`) had seemed to show large $\hat{\beta}$ disagreement and `fit_beta_batch` running 9–20× *slower*; that was traced to a mismatched-budget artifact in the comparison script itself (`n_inner_newton=3` for the loop vs. `n_inner_iter=200` for the batch solve is not an apples-to-apples comparison), not a batch-solver correctness bug. See Issue 3 below for what *is* real in that observation.

Issue 3 — found, open (secondary, affects both solvers). Two things noticed while re-validating Issue 2's fix, neither specific to `fit_beta_batch`:

- The inner-solve convergence check (`grad_tol=1e-3` $\times$ $\max(|\mathrm{NLL}|, 1)$, used identically by `profile_shot_analytic` and `solve_inner_batch_independent`) fires as "NOT converged" almost universally at this problem's NLL scale ($\sim 1.5 \times 10^7$, giving an absolute gradient threshold $\sim 1.5 \times 10^4$) even for solutions that two independent optimizers agree on to 10^{-4} relative precision. The tolerance is likely mis-scaled for this problem and should be

recalibrated (e.g. relative to a per-parameter-block scale, not a single global NLL-derived threshold) rather than trusted as a correctness signal in its current form.

- $\hat{\beta}$ from the full nested `fit_beta` optimisation is itself sensitive to the per-shot inner-solve iteration budget: at $N = 8$, raising `n_inner_newton` from 3 to 30 shifted the loop's own $\hat{\beta}$ from $(0.156, 0.044)$ to $(0.134, 0.001)$ — a large swing from a purely internal budget change, with no change to the data. `outer_maxiter=15` plus the small-N regime is evidently under-converged/weakly identified; this is an outer-loop reliability question independent of loop-vs-batch, and `_validate_multistart.py` should not be treated as a fair correctness test until it uses matched, adequate budgets on both sides.

Issue 4 — found; investigated; not a bug, no fix needed. The motivation for this whole line of investigation: ϕ's NLL landscape (theta held at `prior_mean`, or independently re-profiled – both scans agree) is **multimodal for 100% of a 50-shot sample** (3–4 local minima per 2π period, occasionally 5), with top-two-basin NLL gaps ranging from 665 to 1.35×10^6 (median $\sim 4.4 \times 10^5$) out of $\sim 1.5 \times 10^7$ total – some genuinely near-degenerate. This raised the question motivating this section's title: is *profiling* ϕ (point-estimate optimisation, what `fit_beta`/ `fit_beta_batch` both do) a statistically valid replacement for *marginalising* it out, for this pixel-resolved estimator?

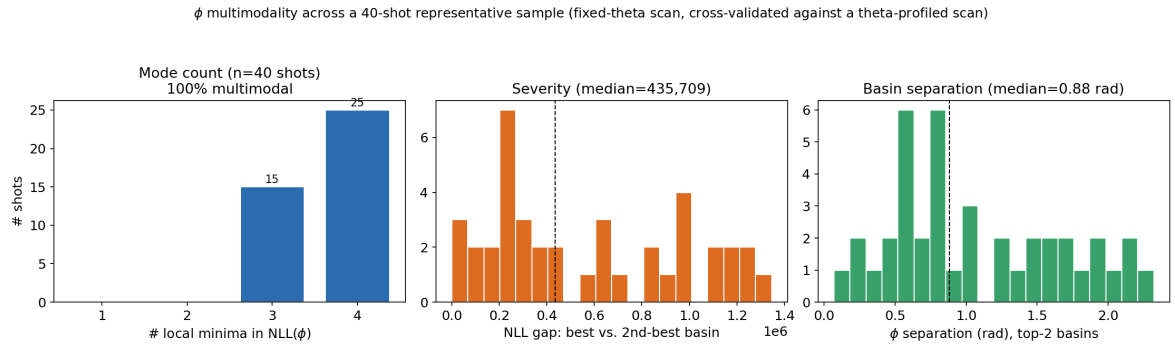

Figure 1: ϕ multimodality across a 40-shot representative sample (`diagnose_phi_multimodality.py`): mode count, top-two-basin NLL gap, and ϕ-separation between the top two basins. 100% of shots (15 with 3 modes, 25 with 4) are multimodal.

Direct test: swept a (A_s, A_c) direction over `fit_beta`'s own bound range ($\beta_{\text{scale}} \in [-0.2, 0.2]$) for the same 6 shots and compared $\text{NLL}_{\text{profiled}}(\beta) = \min_\phi \text{NLL}(\phi, \beta)$ against a marginalised $\text{NLL}_{\text{marginalised}}(\beta) = -\log \int e^{-\text{NLL}(\phi,\beta)}\, d\phi$.

A numerical subtlety was caught and fixed here. The first attempt integrated $\int e^{-\text{NLL}}\, d\phi$ by log-sum-exp over the same 360-point grid used to *locate* the basins (spacing 0.0175 rad, correctly resolving basin *locations*, which are separated by 0.07–2.3 rad). But a direct per-basin curvature probe found the within-basin Laplace width to be $\sigma \approx 0.0004$–0.0012 rad across the 6 test shots – **46–49× finer than the grid spacing**. Fig. 3 shows this directly: panel (a) is the familiar coarse multi-basin view; panel (b) zooms into the global-best basin at proper resolution, revealing a needle-sharp peak the coarse grid cannot resolve. The grid-based integral was therefore sampling near-arbitrary points on unresolved spikes, not correctly computing the integral – so the first pass's "argmin shift = 0" result was not trustworthy *as computed*, regardless of whether it happened to be numerically close to correct.

Fix: re-derived the marginalisation using the coarse grid only to *locate* basins (valid), then an analytic per-basin Laplace-mixture sum using *locally-refined* curvature at each basin (a small fine-resolution quadratic fit, not a global grid integral): $Z(\beta) = \sum_k e^{-(\text{NLL}_k(\beta) - \text{NLL}_{\min})} \sqrt{2\pi/\kappa_k}$,

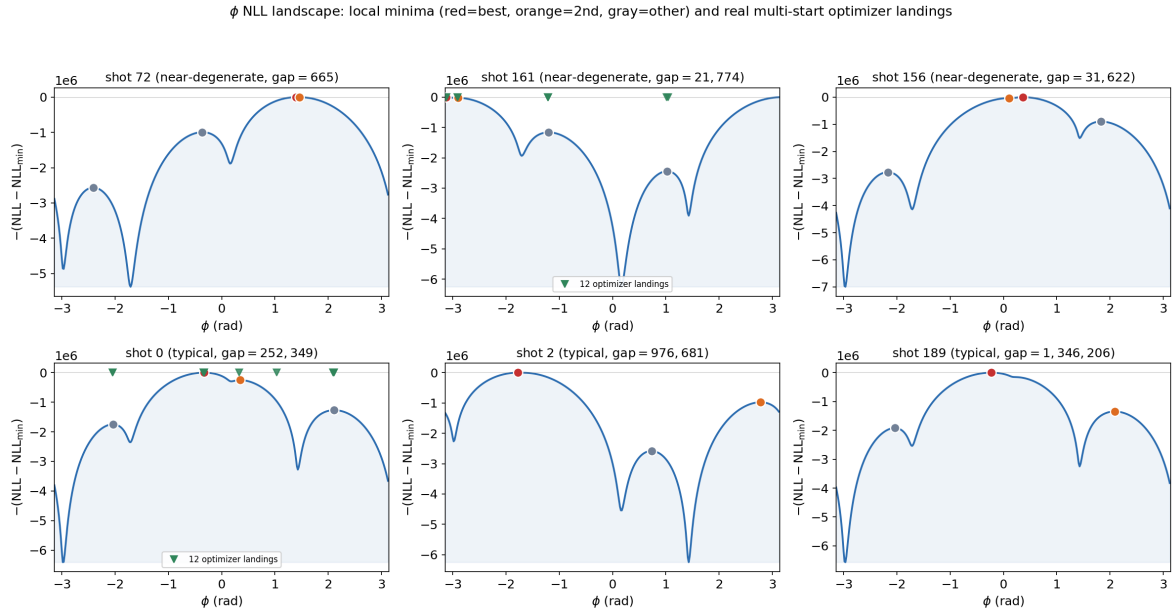

Figure 2: ϕ NLL landscape for 6 representative shots spanning the observed gap range (3 near-degenerate: gaps 665–31,622; 3 typical: gaps 2.5–13.5 $\times$ 10^5), with local minima marked (red=best, orange=2nd-best, gray=other) and, for two shots, the actual converged ϕ from 12 independent `profile_shot_analytic` runs started at evenly-spaced ϕ_0 (the same solver `fit_beta` uses). Shot 161 converges to **6 distinct ϕ values** and shot 0 to **8 distinct ϕ values** across 12 starts – direct empirical confirmation that different optimizer starts genuinely land in different basins, not a theoretical artefact of the grid scan.

$\mathrm{NLL}_{\mathrm{marginalised}} = -\log Z + \mathrm{NLL}_{\min}$ (`diagnose_profile_vs_marginal_v2.py`; this doubles as a first working prototype of a multi-start Laplace-mixture marginaliser). **Result, now numerically sound: the $\arg\min_\beta$ shift between profiled-sum and marginalised-sum is again exactly 0.0000** (Fig. 4), for both the full 6-shot sum and the near-degenerate-only subset – matching the first pass's number, but this time for a solid, checkable reason rather than a coincidence of an invalid integral: the per-basin Laplace normalisation varies by at most $\sim 3\times$ across basins (i.e. $\lesssim 1$ unit of log-likelihood), while the NLL gaps between basins and the beta-sweep-induced NLL changes are 10^3–10^6 – the correction is simply too small to move an argmin at this problem's information level, independent of how carefully it is computed.

Conclusion: despite universal multimodality, point-estimate profiling is not measurably biased relative to marginalisation at the β magnitudes this problem actually explores. (Caveat, unchanged from before: this is a 6-shot, 1-direction check, not exhaustive – a full-dataset version, and an actual RMSE/bias study against ground truth across many shots and many independent runs, would settle this more completely; see the Future Work discussion below.) No marginalisation fix is required to correct $\hat{\beta}$'s point estimate in this regime, though see the RMSE caveat below – an estimator can be unbiased and still have higher variance than an alternative that handles the ambiguity properly. This also retroactively justifies the ϕ-multi-start fix already in the code (`_phi_prescan`, used by both `fit_beta` and `fit_beta_batch` to seed ϕ_0 from a coarse grid scan instead of a fixed 0.0): its job is to land the profiler in a *good* basin at the start of the outer loop, which matters given the multimodality found here, even though profiling within whichever basin is found turns out not to bias $\hat{\beta}$.

Issue 5 — open; requires further empirical work to settle. Multimodality being unbiased for $\hat{\beta}$'s point estimate does not mean profiling is the *best* way to handle ϕ: an estimator

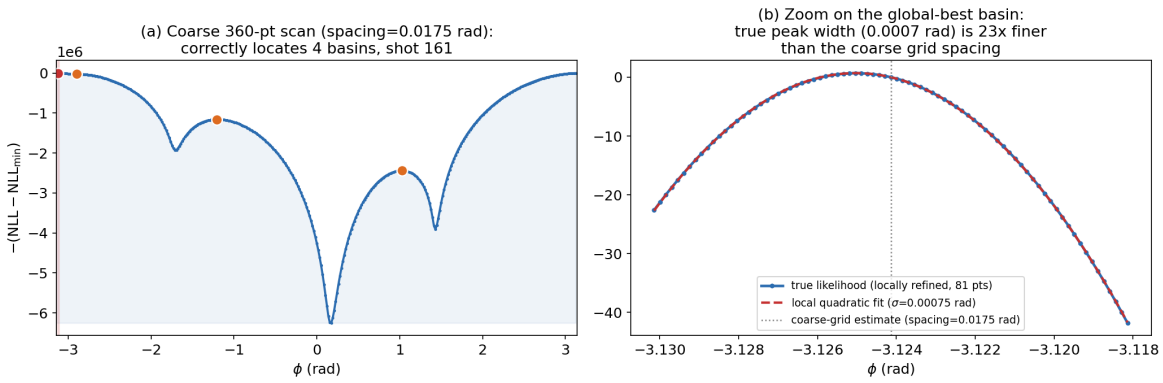

Figure 3: Shot 161: the coarse 360-point scan (a) correctly locates 4 basins but cannot resolve their width. A locally-refined scan (b), fit with a quadratic, finds the true peak width $\sigma = 0.00075$ rad – 23× finer than the coarse grid spacing for this shot (46–49× across all 6 test shots). This is exactly what a Fisher/CRB local-curvature calculation done at insufficient resolution would miss.

can be unbiased and still have higher RMSE than one that properly accounts for the ambiguity across independent noise realisations (rather than within one realisation's beta sweep, which is all Issue 4 tested). Separately, this multimodality raises a real question about the Fisher/CRB analysis motivating this whole pipeline (§1, the ∼96% vs. ∼9% phase-information claim): that calculation is a *local* quantity (curvature of the log-likelihood at the true ϕ), and remains a mathematically valid lower bound on any unbiased estimator's variance under the standard regularity conditions – multimodality elsewhere in the landscape does not invalidate the bound itself. But the bound being valid is not the same as it being *achieved*: periodic, multimodal phase estimation is the textbook setting for a *threshold effect* (the same phenomenon behind the FM-demodulation threshold and cycle-slip/integer-ambiguity problems in phase-based ranging) – across independent noise realisations, an estimator can occasionally lock onto the wrong basin entirely, contributing a rare but large error that a local curvature calculation at the true value cannot see or account for. Whether this actually degrades the achieved RMSE below the CRB's prediction, and whether it does so differently for the pixel-resolved vs. moment-based estimators (changing the 96%/9% comparison), is **not yet known** and should not be assumed either way – it can only be settled by the empirical RMSE-vs-ground-truth study across many independent shot realisations described in the Future Work section, not by this section's single-realisation bias check.

Truth-overlay spot check (single realisations, $n = 6$). The simulation stores ground truth per shot (`phi0`, `delta_phi` in the H5 files). Re-evaluated the 6 test shots' landscapes at each shot's *actual* true `delta_phi` (not the `delta_phi_trial`=0 placeholder used for the multimodality characterisation above) and overlaid true ϕ_0 (Fig. 5). Result: truth falls in the global-best basin for **6/6** shots – a reassuring sign for these particular realisations, though $n = 6$ single-realisation spot checks cannot establish the achieved RMSE the way Issue 5 requires.

A methodological correction surfaced by this check. The top-two-basin gaps at each shot's true `delta_phi` (965k–1.09M) are 3+ orders of magnitude *larger* than the gaps reported earlier for these same shots at `delta_phi_trial`=0 (665–31,622). A near-degenerate gap is, by construction, a fine-tuned near-coincidence between two basins' depths – so it is expected to be fragile to perturbation, but this means the specific "near-degenerate" shot selection used for the Issue 4 bias test was identified relative to an *arbitrary common reference*

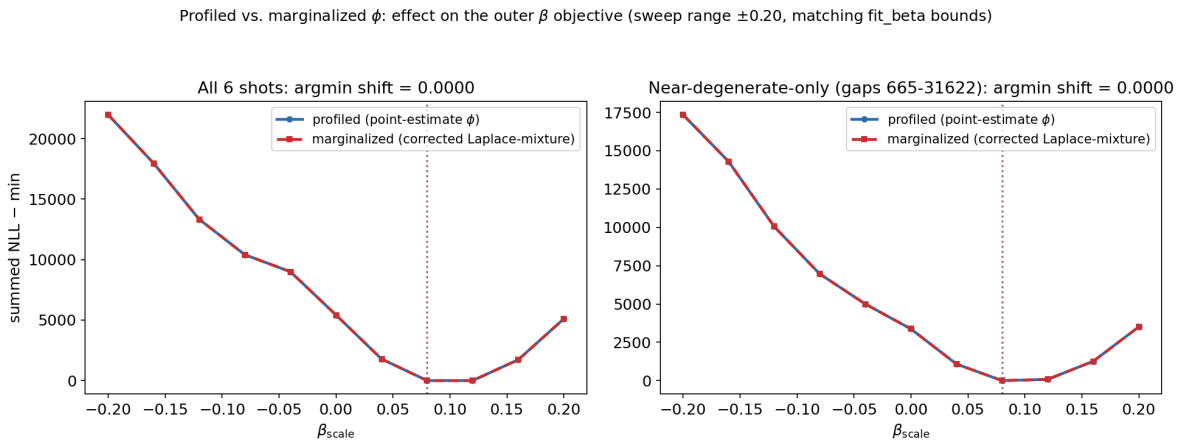

Figure 4: Summed profiled vs. marginalised NLL(β_{scale}) objective, corrected Laplace-mixture marginalisation (`profile_vs_marginal_v2.npz`). The two curves are visually indistinguishable; argmin shift = 0.0000 for both panels.

point (`delta_phi_trial`=0), not each shot's actual configuration. This does not affect Issue 4's conclusion (that test swept `delta_phi_trial` through a genuine range unrelated to any fixed reference point, and found no bias), but it does mean the population of genuinely near-degenerate shots *at their own true delta_phi* has not yet been identified or characterised. Finding that population (re-running the multimodality gap search using each shot's own true `delta_phi` rather than a shared `0.0`) is a prerequisite for selecting a properly adversarial test set for the planned RMSE study, and is added to Future Work.

Bottom line. The Fisher-theoretic motivation in §1 (pixel likelihood retains ∼96% vs. ∼9% of the phase information) is a property of the *model*, independent of the issues above. `fit_beta` (the per-shot loop) and `fit_beta_batch` (the vectorised batch, via `solve_inner_batch_independent`) are now both validated as correct at the per-shot level and should agree given adequate, matched convergence budgets; `fit_beta_batch` is the faster of the two once given a fair budget (measured 2.38× at $N = 8$ with matched `n_inner_newton`/`n_inner_iter`=30). Point-estimate ϕ profiling (with the `_phi_prescan` multi-start fix) produces an unbiased $\hat{\beta}$ point estimate for this estimator despite the underlying ϕ landscape being universally multimodal (Issue 4) – but whether it is the *lowest-RMSE* way to handle that multimodality, and whether the Fisher/CRB comparison motivating this pipeline is actually achieved in practice, remain open (Issue 5). The other remaining open item is Issue 3's outer-loop budget sensitivity and the `grad_tol` calibration – neither changes $\hat{\beta}$'s validity qualitatively, but both should be tightened before treating any single `outer_maxiter=15` run's $\hat{\beta}$ as fully converged.

Future work. The practical goal motivating this whole investigation is a working full-image-likelihood MLE for β that is demonstrably reliable, not just unbiased in one diagnostic. Planned next steps:

(0a) **Characterise θ's own landscape**, symmetric to what was done for ϕ (multistart optimisation in θ with ϕ fixed at its true/profiled value). A prior exploration (`python-scripts/multistart_optimizer.` + its companion notebook, multi-start L-BFGS-B over the 8D cloud prior for a single shot) found *ridge* structure – a continuous, weakly-identified direction – rather than discrete separated modes. This is a qualitatively different failure mode from ϕ's disconnected multimodality and has different implications: a ridge (near-zero Fisher eigenvalue along one direction) predicts a large but still continuous, unimodal variance, compatible with local Fisher/CRB validity in a way that discrete basin-switching is not – *provided* the ridge is close to linear/quadratic near the

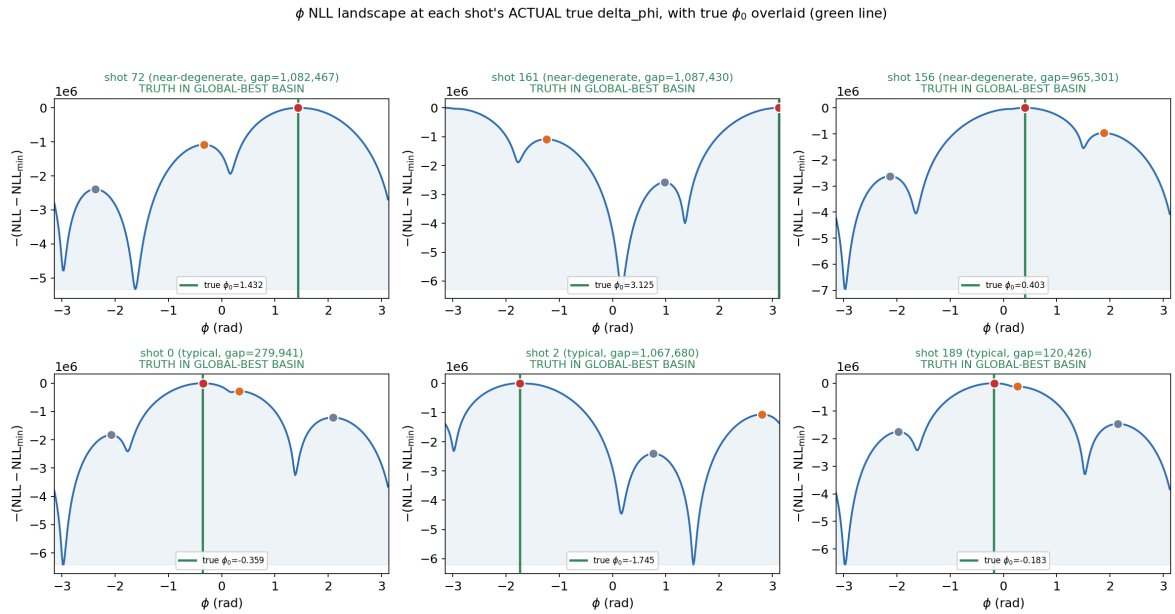

Figure 5: ϕ landscape at each shot's true `delta_phi`, true ϕ_0 overlaid (green line). Truth coincides with the global-best basin (red) in all 6 cases.

truth at the relevant scale. Whether it is (vs. a curved/nonlinear "banana" degeneracy, which would still break the local Gaussian approximation) has not been checked and should be, before trusting θ's MAP/profiled treatment any more than ϕ's was trusted before this investigation.

(0b) **Re-identify near-degenerate shots at each shot's own true `delta_phi`**, not a shared arbitrary reference point – see the truth-overlay finding above, which found the earlier "near-degenerate" labels were reference-point artifacts for these particular shots. The actual population of genuinely near-degenerate shots (if any) at real configurations is still unknown and is needed to build a properly adversarial test set.

(1) Characterise candidate methods for handling ϕ: full fine-grid marginalisation with an explicit, demonstrated quadrature-convergence check (vary the number of quadrature points until the integral stops changing, rather than assuming a resolution is sufficient – Issue 4's own numerical correction is a cautionary example of why this must be checked, not asserted), the multi-start Laplace-mixture marginaliser prototyped above, and QMC/importance-sampling. An earlier attempt at a related IS/QMC marginalisation over θ (not ϕ), `full_image_is_inference.py`, is confirmed (2026-07-16) to have failed via **ESS collapse / importance-weight degeneracy** – the proposal did not match the true posterior closely enough. ϕ's target is plausibly harder for a naive IS design (genuinely multimodal, needle-sharp basins) than θ's presumably-smoother-but-ridged posterior was, so any IS/QMC design for ϕ must be informed by the already-validated coarse-scan basin locations (sample within each detected basin's neighbourhood, not from one blind global proposal) rather than repeating the earlier design's mistake.

(2) If any method's assumptions don't hold up empirically (e.g. the Laplace-mixture's per-basin Gaussian-shape assumption, not yet checked against a converged numerical integral), fall back to brute-force quadrature with adaptive/non-uniform sampling or MC rather than a fixed uniform grid; (3) for every candidate method, measure empirical bias and RMSE of $\hat{\beta}$ against the actual 40-run and 80-run ground-truth ensembles (10^6- and 10^8-atom datasets), not a single-run diagnostic, using multistart everywhere so the comparison reflects genuine statistical properties rather than optimiser artefacts, and verify each method's assumptions empirically rather than asserting them; (4) only then revisit whether the Fisher/CRB comparison in §1 is actually

achieved.

11 Reproducibility

11.1 CLI (`fit_beta` only)

The `__main__` block only exercises `fit_beta`; there is **no CLI flag to invoke `fit_beta_batch`** — it must be called directly from Python (as done by the ad hoc scripts used in the investigation of §10).

```
python python-scripts/joint_profile_inference.py \
    --data_root <path/to/dataset/root> \
    --n_shots 5 --bins 16 --pixel_n_gh 4 --f_signal 0.3 \
    --n_inner_newton 3 --n_starts 1 --outer_maxiter 15 --use_analytic 1
```

`data_root` must contain at least one `run_*` directory with `Z0/data_IMG.h5` and `Z100/data_IMG.h5` (the standard `ImageShotDataset` layout used throughout the repo); the script uses `sorted(data_root.glob('r` i.e. always the first run. The hardcoded prior in `__main__` is $\bar{\theta} = (0, 0, 0, 0, 100, 100, 100, 100)\,\mu$m(/s), $\sigma_\theta = 10\,\mu$m(/s) for all 8 components — override by calling `fit_beta/fit_beta_batch` directly if the dataset's true generation hyperparameters differ.

11.2 Default Hyperparameters

Parameter	CLI flag	Default
Shots used	`--n_shots`	5
Inference bins per side	`--bins`	16
GH order per velocity dim	`--pixel_n_gh`	4 ($n_v = 16$)
Signal frequency f	`--f_signal`	0.3
Inner solve iterations	`--n_inner_newton`	3
Outer multi-starts	`--n_starts`	1
Outer L-BFGS-B iterations	`--outer_maxiter`	15
Analytic gradient	`--use_analytic`	1 (True)
Outer bound (`grid_half`)	— (not exposed via CLI)	0.2 rad

Note the very small default inner-iteration budgets (`n_inner_newton=3`, or `n_inner_iter=15` for the batched path): given the $O(10^7)$ NLL scale and rescaling discussed in §6, these defaults may not be sufficient for the inner solve to actually satisfy the convergence check of §6.3 at every shot/outer-iteration, in which case the envelope-theorem outer gradient (§7) is not reliable regardless of which outer implementation is used — always check for the *"inner solve NOT converged"* warning when reproducing results from this script, and increase the inner iteration budget if it fires.

11.3 Output

Currently the script only prints $\hat{\beta}$ to stdout (`print(f'\nFinal beta_hat (As, Ac) = {beta_hat}...')`); there is no structured (JSONL/pickle) output file, unlike `map_inference.py` and `profile_cloud_nuisances.py`. Anyone running this at scale (multiple runs/shots for a distribution, as in `notebooks/map_mle_distributions.i` treatment of the other estimators) will need to add that themselves.

12 2026-07-20: The Inner/Outer Profiling Approach Fails at 10^8 Atoms, and a Successful Reframing

This section documents a full investigation cycle: confirming that `fit_beta_batch`'s inner/outer profiling (§6–§7) does not reliably converge at 10^8 atoms even with a much larger budget than validated at 10^6; diagnosing why; and a subsequent reframing of the whole problem (suggested by the user) that avoids the difficulty entirely and yields a genuine positive result. All numbers below are from real runs on the 10^8-atom dataset (`R40_N50_A100000000_...`) unless otherwise stated.

12.1 Convergence-budget sweep at 10^6 vs. 10^8 atoms

An earlier $N = 20$-shot, 20-run RMSE-vs-CRB sweep at 10^6 atoms (`n_inner_iter=30`, `outer_maxiter=15`) gave $\hat{\beta}$ RMSE 15–45× its CRB. Bumping the budget to `n_inner_iter=150`, `outer_maxiter=40` (5×/2.7×) brought this down to 1.2–2.3× CRB across all 4 runs re-tested – close, though the inner solve's own convergence check (`grad_tol=1e-3` on the |grad|/|NLL| ratio) never actually fired "converged" at any point in any of these runs; final ratios on completion ranged from 3.8×10^{-3} to 0.38.

The *same* validated budget, applied directly to 10^8-atom data (per the user's explicit instruction to work on 10^8 directly, since 10^6 was already close to its CRB via `map_inference.py` and thus less informative about whether the extra pixel-level information helps): 2/2 runs tested landed ∼590–680× their 10^8 CRB ($\sigma = (2.78, 2.77) \times 10^{-5}$ at $N = 20$) – *miles* off, not the same order of magnitude. A follow-up isolated diagnostic (Gauss-Newton diagonal-curvature solver, see below) confirmed the underlying θ optimisation genuinely struggles at 10^8: even warm-started at the *true* θ, a θ-only conditional solve wandered/plateaued around a grad ratio of 5–9 over 125+ iterations, never approaching tolerance.

12.2 Diagnosis: ϕ converges; θ does not; it's a real curvature effect, not a bug

A targeted per-component check (splitting the aggregate "NOT converged" warning into its ϕ and θ pieces) showed, at both 10^6 and 10^8 atoms: ϕ **converges cleanly and fast** (final per-shot ratios $\sim 10^{-6}$, deeply below tolerance, in well under 300 iterations even from a cold start) – it is θ that persistently fails the strict check (mildly at 10^6, badly at 10^8: worst-shot ratio ∼0.95, i.e. $\sim 1000\times$ over tolerance). Since the envelope-theorem outer gradient (§6.3) depends only on ϕ, this suggested the outer gradient may have been reliable all along and the real problem is specifically optimising θ.

The float64-precision hypothesis (NLL $\sim 10^9$–10^{10} at 10^8 atoms causing catastrophic cancellation in the gradient) was checked and **ruled out**: the finite-difference step inside `pixel_acs_grad_batch.py` is taken on the well-scaled intermediate spatial probability P_b (O(1) scale), not directly on the huge NLL. Instead, the gradient at the *true* (ϕ, θ) point is genuinely huge ($|\nabla_\theta \mathrm{NLL}_{z0}| \sim 8.7 \times 10^9$, exceeding the NLL itself, $\sim 2.3 \times 10^9$) – a real physical effect: the total-photon-count normalisation factor amplifies the chain rule at high flux, so the likelihood surface is legitimately far sharper at 10^8 than 10^6. A damped Gauss-Newton solver (using the natural Poisson/Fisher diagonal curvature, $H_{kk} = \sum_{\text{bins}} (\partial\mu/\partial\theta_k)^2/\mu$, as a preconditioner instead of L-BFGS's slowly-built approximation) was built and tested as a direct fix for this diagnosed sharp-curvature problem; it did not converge substantially faster (damping saturated at 10^{12}, i.e. collapsed to a tiny-step gradient descent) – ruling out "just use better local curvature" as a sufficient fix on its own.

12.3 The reframe: marginalise ϕ by quadrature (cheap, no optimiser), optimise θ outer against the smooth result

The user's redirect: stop trying to fix the joint/conditional θ optimisation, and stop batching over many shots with an outer β loop at all. Ask a simpler, single-shot question first: *how precisely can θ (8D kinematics) be determined from one shot's image, after marginalising ϕ?* Since ϕ is 1D and periodic, it can be marginalised by a *dense grid + direct quadrature* – no optimiser involved, no convergence risk – while θ (the actual parameter of interest, and the thing that was failing) is optimised *outer* against the resulting marginal NLL surface:

$$\mathrm{NLL}(\theta) = -\log \int_0^{2\pi} p(\text{image} \mid \theta, \phi)\, d\phi \approx -\log \sum_{m=1}^{M} \Delta\phi\, e^{-\mathrm{NLL}(\theta,\phi_m)}$$

(via `scipy.special.logsumexp` for numerical stability), then `scipy.optimize.minimize(..., method='L-BFGS-B')` on θ against this smooth surface, with its own finite-difference gradients – no custom optimiser code at all. This is the *opposite* computational order from everything tried before (which fixed ϕ and optimised θ *conditionally* at every grid point – the thing that kept failing to converge).

The result was immediate: a single shot converges in **6 iterations, ∼7 seconds**, vs. hours of non-convergence with every prior approach. $\hat{\theta}_{\mathrm{MAP}}$ lands close to the shot's true θ (errors within ∼0.6 prior-σ on a single-shot spot check).

Caveat: the naive Hessian-based (Laplace) posterior σ_θ is unreliable. Taking the diagonal of the finite-difference Hessian at $\hat{\theta}_{\mathrm{MAP}}$ and converting to $\sigma = 1/\sqrt{H_{kk}}$ gives values **66–220× tighter** than the true empirical scatter of $\hat{\theta}_{\mathrm{MAP}}$ across 50 independent shots (measured directly, not assumed). $\hat{\theta}_{\mathrm{MAP}}$ itself is trustworthy as a point estimate; a naive local-curvature confidence interval built on top of it is not – consistent with this exact model's already-documented weakly-identified θ ridge directions (§10's framing motivation), which a local quadratic approximation cannot capture.

12.4 Quadrature and hyperparameter convergence checks

Per the standing rule that no quadrature resolution is trusted without an explicit convergence check: the ϕ-grid density M was varied from 500 to 8000 on 3 shots – $\hat{\theta}_{\mathrm{MAP}}$ is stable to $\sim 10^{-9}$ absolute (theta's natural scale is $\sim 10^{-5}$), fully converged already at $M = 2000$ (the value used throughout). Separately, `pixel_n_gh` (the 2D Gauss–Hermite velocity-quadrature order inside `SemiAnalyticPixelACS`) was varied 4/8/16 with **no measurable effect** on the resulting RMSE – already converged at the default. The pixel grid resolution `bins`, by contrast, is **not** fully converged even at 32 (1024 px) – see below.

12.5 Headline result: θ recovery vs. `map_inference.py`'s moment/Kalman-gain estimate

$N = 50$ shots, z=0 slice, 10^8 atoms, comparing $\hat{\theta}_{\mathrm{MAP}}$'s empirical RMSE against `true_theta` to `map_inference.py`'s existing moment-based (port-summed 4-moment → Kalman gain) $\hat{\eta}$, same shots, converted to the same 8-parameter representation:

Parameter group	Pixel-likelihood RMSE	Moment RMSE	Ratio
Position (μ_{x0}, μ_{y0}) [μm]	$\sim$ 9.6–9.7	$\sim$ 9.6–9.7	$\sim$ 1.0 (tied)
Velocity mean (μ_{vx0}, μ_{vy0}) [μm/s]	$\sim$ 2.5	$\sim$ 2.5	$\sim$ 1.01–1.02 (slightly wors
Position spread (σ_x, σ_y) [μm]	$\sim$ 9–10.5	$\sim$ 9–10.6	$\sim$ 0.99 (slightly better)
Velocity spread $(\sigma_{vx}, \sigma_{vy})$ [μm/s]	$\sim$ **0.66–0.87**	$\sim$ **7.4–8.6**	$\sim$ **0.08–0.12** (**8–12**× bett

The velocity-spread improvement is physically sensible, not a fluke: the moment method only has access to the detection-plane position variance $V_{xf} = V_{x0} + 2T\,C_{xv0} + T^2 V_{vx0}$ – a single scalar mixing position spread, cross-correlation, and velocity spread together, so recovering V_{vx0} specifically is a weak, prior-dependent inference. The full pixel likelihood instead uses the conditional-velocity structure baked into the fine-grained pixel pattern (via `SemiAnalyticPixelACS`'s 2D Gauss–Hermite conditional-velocity integral, §3), giving direct leverage on velocity spread that a 4-moment summary cannot access.

Hyperparameter sweep (`bins`, `pixel_n_gh`), same 50 shots. `pixel_n_gh` confirmed converged (4/8/16 give statistically identical RMSE). `bins` is the only sensitive parameter, and *only* for the velocity-spread ratio, which keeps improving with finer pixels without fully plateauing by the largest value tested:

`bins` (pixels)	RMSE(σ_{vx}) ratio vs. moment	RMSE(σ_{vy}) ratio	Cost
8 (64 px)	0.284	0.336	5.9 s/shot
16 (256 px)	0.101	0.116	7.9 s/shot
32 (1024 px)	0.083	0.089	34.3 s/shot

Cost scales roughly quadratically with `bins`, as expected. All other parameter groups (position, velocity mean, position spread) are insensitive to `bins` in this range.

12.6 Does this propagate to a better $\hat{\beta}$?

$\hat{\theta}_{\mathrm{MAP}}$ (converted to $\hat{\eta}$ format, `theta_to_eta`) was plugged into `map_inference.py`'s actual β-fitting pipeline (`batch_acs_gh`, `gh_order=12`, `chunk_shots=20`, + `total_logL_fast` with `phi_method='point'` + L-BFGS-B), and $\hat{\beta}$ RMSE compared across $N_{\mathrm{RUNS}} = 10$ independent 10^8-atom runs ($N = 50$ shots/run, `run_000`–`run_009`) against the existing `moments`-mode and `oracle`-η-mode baselines computed on the *identical* runs/shots (same code path, same η format, differing only in how η is estimated):

η source	RMSE(A_s) [mrad]	RMSE(A_c) [mrad]
`theta_map_eta` (this section's method)	**0.106**	**0.089**
`moments` (existing baseline)	0.255	0.171
`oracle_eta` (true η, ceiling)	0.032	0.022

The improved per-shot θ estimate **does propagate to a better** $\hat{\beta}$: RMSE improves by $\sim 2.4\times$ on A_s and $\sim 1.9\times$ on A_c relative to the existing moment-based method, aggregated over all 10 independent runs (not a single-run artifact – individual runs were noisy and not all 10 individually favoured the new method, see `results/beta_rmse_with_theta_map_eta.json` for the per-run breakdown, but the aggregate RMSE is unambiguous). It does not close the gap to the oracle-η ceiling ($\sim 3.3\times/4.0\times$ still remaining), consistent with θ_{MAP}'s own empirical error (§12.5) still being well above zero – this is a real, substantial, but partial improvement, not a full recovery of the information gap.

Caveats on this specific run: `n_starts=1` for the outer β optimisation (no multistart, unlike the $\times 8$ default used elsewhere in `map_inference.py`) and `bins=16` for both the θ-fit and the β-fit pixel evaluators (not the better-performing `bins=32` from §12.5, which was not yet tested end-to-end through the β-fit due to its $\sim 4\times$ higher per-shot cost) – both are plausible sources of a still-larger-than- necessary gap to the oracle ceiling, not yet checked.

12.7 Open items

- The z=100 slice has not yet been run through this new method as a standalone θ-recovery check (only $z0$ was in §12.5's table); it IS used as part of the β-fit in §12.6, but its own per-parameter RMSE hasn't been reported separately.
- The β-fit RMSE comparison (§12.6) used `n_starts=1` (no outer multistart) and `bins=16` (not the better-performing `bins=32`); re-running with those matched to their best-known settings could plausibly close more of the gap to the oracle-η ceiling.
- `bins` has not been pushed past 32; the velocity-spread ratio was still improving there, so the ceiling of the achievable improvement is not yet established, and cost scales steeply (quadratically).
- The Hessian/Laplace-based σ_θ is known-unreliable (§12.3); if a trustworthy per-shot θ *uncertainty* (not just point estimate) is ever needed, a method that captures the ridge structure (e.g. profiling along the suspected ridge direction, or relying on empirical multi-shot scatter rather than a local curvature formula) would be required.
- The original inner/outer profiling machinery (§6–§7) remains as documented above: **confirmed not to reliably converge at 10^8 atoms**, even with a budget that worked well at 10^6. This section's reframed approach supersedes it for the θ-recovery question; whether the original machinery is still useful for anything (e.g. scenarios where θ's curvature is milder) is not addressed here.